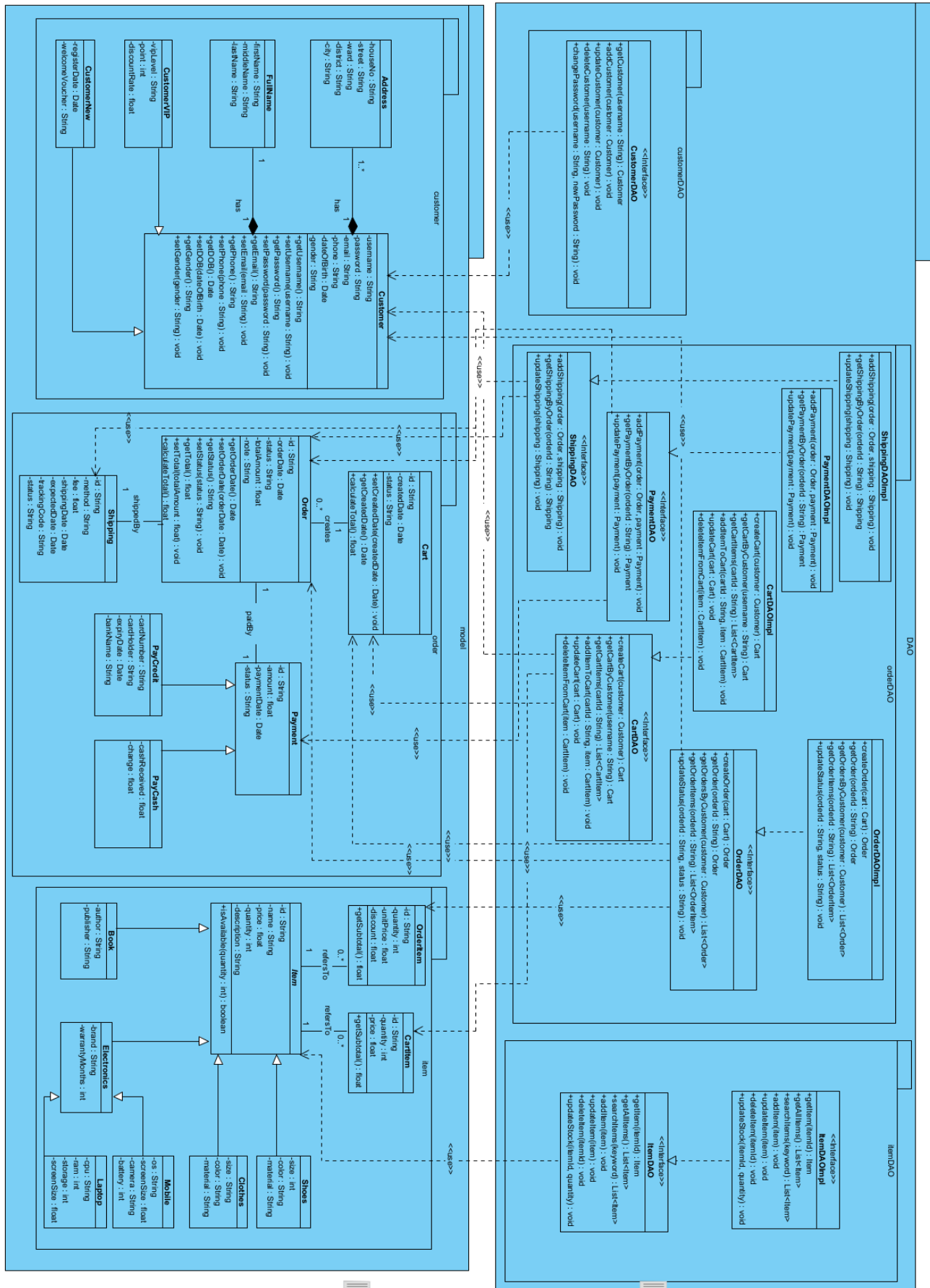




## E-commerce System: Design Class Diagram (DAO & MVC)

### 1. DAO and MVC

- **MVC** splits the system into three layers. The Model holds data and business rules, the View is what the user sees, and the Controller takes each request, calls the Model or DAO, and passes the result back to the View.
- **DAO** (Data Access Object) is a separate layer that reads from and writes to the database. Each DAO is an interface plus an implementation class (...Impl), so model classes never contain database code.
- A «use» dependency means a DAO method takes or returns an object of that model class.

### 2. Package structure

The DAO package mirrors the model package, so each group of classes has its own DAO package.

| Model package | Classes                                                                      | DAO package | DAOs                                                           |
|---------------|------------------------------------------------------------------------------|-------------|----------------------------------------------------------------|
| customer      | Customer, Address, FullName, CustomerVIP, CustomerNew                        | customerDAO | CustomerDAO, CustomerDAOImpl                                   |
| order         | Cart, Order, Payment, PayCredit, PayCash, Shipping                           | orderDAO    | CartDAO, OrderDAO, PaymentDAO, ShippingDAO (each with an Impl) |
| item          | Item, Book, Electronics, Mobile, Laptop, Shoes, Clothes, CartItem, OrderItem | itemDAO     | ItemDAO, ItemDAOImpl                                           |

Some classes don't need their own DAO:

- **Address and FullName** are parts of Customer (composition), so they are saved together with it.
- **Subclasses** such as CustomerVIP, PayCredit or Laptop share their parent's DAO.
- **CartItem and OrderItem** are handled by CartDAO and OrderDAO.

### 3. DAOs and operations

get methods look records up by key (id or username), while add and update methods take the model object itself. Each ...Impl class implements every operation of its interface.

| DAO         | Operations                                                                                                                                                                                                     | Uses                             |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| CustomerDAO | getCustomer(username) : Customer<br>addCustomer(customer) : void<br>updateCustomer(customer) : void<br>deleteCustomer(username) : void<br>changePassword(username, newPassword) : void                         | Customer                         |
| CartDAO     | createCart(customer) : Cart<br>getCartByCustomer(username) : Cart<br>getCartItems(cartId) : List<CartItem><br>addItemToCart(cartId, item) : void<br>updateCart(cart) : void<br>deleteItemFromCart(item) : void | Customer, Cart, CartItem         |
| OrderDAO    | createOrder(cart) : Order<br>getOrder(orderId) : Order<br>getOrdersByCustomer(customer) : List<Order><br>getOrderItems(orderId) : List<OrderItem><br>updateStatus(orderId, status) : void                      | Customer, Cart, Order, OrderItem |
| PaymentDAO  | addPayment(order, payment) : void<br>getPaymentByOrder(orderId) : Payment<br>updatePayment(payment) : void                                                                                                     | Order, Payment                   |
| ShippingDAO | addShipping(order, shipping) : void<br>getShippingByOrder(orderId) : Shipping<br>updateShipping(shipping) : void                                                                                               | Order, Shipping                  |
| ItemDAO     | getItem(itemId) : Item                                                                                                                                                                                         | Item                             |

| DAO | Operations                                                                                                                                                                              | Uses |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|     | getAllItems() : List<Item><br>searchItems(keyword) : List<Item><br>addItem(item) : void<br>updateItem(item) : void<br>deleteItem(itemId) : void<br>updateStock(itemId, quantity) : void |      |

#### 4. Main business rules

**Cart:** Every customer has one cart, created when they register (`createCart`). Before an item goes into the cart, the system checks stock with `Item.isAvailable(quantity)`. The cart total is the sum of each line's `CartItem.getSubtotal()`.

**Placing an order:** `createOrder(cart)` turns the cart items into order items. Each `OrderItem` keeps the unit price and discount at the time of purchase, so later price changes don't affect past orders. Once the order goes through, the cart is emptied and stock is reduced with `updateStock`.

**Pricing and discounts:** VIP customers get a discount based on `discountRate`, and new customers can use their `welcomeVoucher` on their first order. `Order.calculateTotal()` adds up the line subtotals, subtracts any discount and adds the shipping fee.

**Order status:** An order moves from Pending to Confirmed, Shipping and finally Completed, or becomes Cancelled if the customer cancels before it ships. Every change goes through `updateStatus(orderId, status)`.

**Payment and shipping:** Each order has one Payment, either cash (`PayCash`, which records the amount received and the change given) or card (`PayCredit`). Shipping stores the method, fee, expected delivery date and a tracking code the customer can use to follow the package.